Zero-Ping: A Packet-Loss-Resilient Neural Codec for Real-Time Voice Communication

May 29, 2026

Luca Brunetti 1967993

Abstract

Who has not experienced the frustration of packet loss while playing online games? This project addresses packet loss concealment for real-time mono 16 kHz speech transmission in a ”neural way”. Zero-Ping is an end-to-end trained neural audio codec based on an RVQ-GAN architecture, designed to reconstruct the audio waveform even when some transmitted packets are lost.

1. Introduction

Unreliable transmission conditions are common in online gaming voice chat and, more generally, in real-time communication scenarios. Existing neural audio codecs are mainly designed to optimize compression efficiency and perceptual reconstruction quality under reliable transmission. However, in real-time communication, lost packets cannot be recovered by retransmission without introducing additional delay. As a result, packet loss may permanently remove part of the transmitted representation, severely compromising speech intelligibility.

In this project, I designed and trained Zero-Ping, a neural speech codec based on an RVQ + Transformer architecture, following the RVQ-GAN paradigm used by modern neural audio codecs. The natural transmission cycle is that the encoder receives a clean waveform, tokenizes it, and sends the resulting packets. During transmission, some packets can be lost. As a consequence, the decoder either receives no information for the missing time interval or simply continues decoding from the next packet that arrives. Realistical packet loss patterns do not follow a purely random distribution, such as an iid Bernoulli process. Instead, they typically occur in bursts over time. To simulate this behavior, i implemented the **Gilbert-Elliott model** (1). This model describes packet loss through a two-state Markov process, where the network alternates between a good state and a bad state. The persistence of the bad state makes consecutive packet losses more likely, thus reproducing bursty loss behavior:

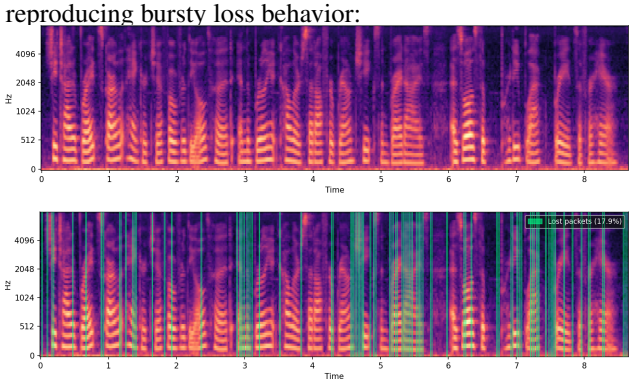

Figure 1. Top-Figure: Spectrogram of the original speech signal. *Bottom-Figure:* Packet-loss pattern generated with the Gilbert–Elliott model, with an overall loss rate of 17.9%. Green vertical lines mark lost packets, where each packet corresponds to a 15 ms audio frame.

Crucially, no pre-corrupted dataset is needed: the *Gilbert-Elliott* simulator is applied online, directly in the latent domain after the RVQ quantizer, at every training step. This mirrors the actual transmission scenario: the encoder quantizes and “sends” the latent frames, the channel drops some of them, and the decoder must reconstruct from what arrives. This ensures that the model is exposed to a virtually unlimited variety of loss scenarios during training, naturally covering the full range of burst lengths and loss rates that the Gilbert-Elliott can produce, without any offline data augmentation pipeline. Zero-Ping handles these time holes by predicting the missing latent vectors before they reach the decoder. Thus, it is capable of reconstructing speech, even in critical connection scenarios, without compromising intelligibility.

2. Related works

Although existing neural audio codecs are not primarily designed for packet loss concealment, **EnCodec** (2) and **SoundStream** (3) served as the main architectural refer-

Email: Luca Brunetti <brunetti.1967993@studenti.uniroma1.it>.

Deep Learning and Applied AI 2026, Sapienza University of Rome, 2nd semester a.y. 2025/2026.

ences for Zero-Ping. Despite targeting different bitrate ranges and sampling frequencies, both systems inspired the real-time encoder–quantizer–decoder structure.

Additional implementation details were inspired by **Improved RVQGAN** (4), including RVQ codebook initialization, the use of Snake activations to improve waveform reconstruction, and several training design aspects.

3. Architecture and Training Methodology

The project's target is voice chat, so we operate at mono 16kHz. I decided to operate at a low bitrate (6kbps). The design is driven by two main priors. First, during communication we know where and when a packet is missing: transmission protocols such as **RTP** (5), provides sequence numbers and timestamps. This enables informed packet loss concealment rather than blind waveform prediction. Second, the overall system must satisfy the real-time constraint of one-way delay below 150ms, following the **ITU-T G.114** (6) standard. The system consists of four components (implementation details are documented in the repository). A causal convolutional **encoder** maps the waveform to one 128-dimensional latent frame every 240 samples, corresponding to 15ms of audio. A 9-stage **RVQ quantizer**, with codebooks of size 1024, so $66.7 \times 10 \times 9 \approx 6\text{kbps}$. A **Transformer** that operates in the quantized latent domain: missing frames are replaced by learned mask embeddings, while a binary loss mask (0 for lost, 1 for received packets) tells the model which positions are reliable during local attention Each lost frame is estimated from a context window of 12 past and 2 future frames, adding 30ms of lookahead. Finally, a causal **decoder** reconstructs the waveform from the repaired latent sequence using Snake activations, optimal for reconstructing periodic structures, following RVQGAN-style audio codecs (4). The critical part on designing the architecture was the Real-Time/reconstruction quality tradeoff. Each component was tested with the **Real-Time-Factor benchmark (RTF)** before trained in order to stay inside the 150ms constraint counting up the 30ms lookahead and another 30ms of trasmission delay.

The model was trained for approximately 1M steps. Training data covers a variety of speakers, acoustic conditions, and microphone setups (LibriTTS, VCTK ecc...). As said, no offline data augmentation or pre-corrupted dataset is used: packet-loss degradations are simulated entirely online via the Gilbert-Elliott model, applied directly in the latent domain after the RVQ quantizer; mirroring the actual codec deployment logic. Training proceeds in three stages. In the first, the encoder, RVQ quantizer, and decoder are trained without the Transformer or packet-loss simulation, establishing a stable latent space. In the second, the codec is frozen and only the Transformer is trained, with the Gilbert-Elliott simulator active. In the third,both are unfrozen and fine-tuned end-to-end together to act as a single component. Throughout training, the codec acts as the generator in a GAN, adversarially trained against a MS-STFT discriminator, encouraging perceptually sharper reconstructions than purely reconstruction-based objectives (2). For the losses check the Appendix-5.

4. Evaluation

Real-Time Performance: Benchmarks on an NVIDIA RTX 3050 Laptop GPU show that Zero-Ping exhibits a nearly fixed per-call overhead of approximately 70–75 ms, largely independent of the processed chunk duration. As a result, short chunks are dominated by invocation overhead rather than by the intrinsic computational cost of the model. At 75 ms, Zero-Ping reaches real-time performance on average, with RTF $= 0.996$, making this the minimum chunk size for average real-time execution on this hardware.

Table 1. (6 kbps). RTF < 1.0 indicates real-time capability.

Chunk	RTF	$\pm$ std	Mean (ms)	Real-Time
60 ms	1.3164	0.2465	79.0	✗
75 ms	0.9960	0.1964	74.7	✓
90 ms	0.8173	0.1513	73.6	✓

Reconstruction Quality: The most common benchmark for quality is the **MUSHRA**, but since for me it was unpractical, i opted for objective metrics: STOI and PESQ. Figure 2 shows an example on a single audio signal.

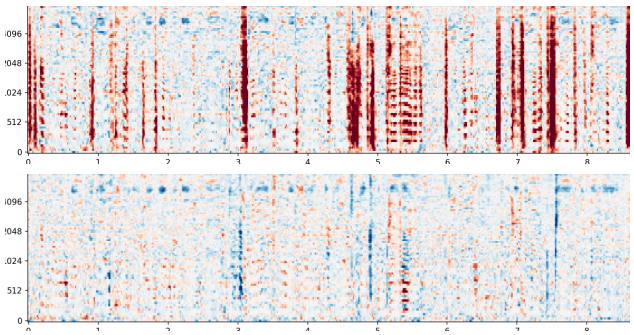

Figure 2. Spectral error maps showing the effect of packet loss on the same speech audio shown in Figure 1. Both examples use the same Gilbert-Elliott packet-loss pattern. *Top:* reconstruction without the repair module, where packet losses introduce strong localized spectral distortions and a lower intelligibility score, STOI $= 0.841$. *Bottom:* Zero-Ping reconstruction with latent repair, which substantially reduces packet-loss artifacts and improves intelligibility, reaching STOI $= 0.942$. Color intensity represents the spectral difference in dB between the reconstructed and reference signal.

Final evaluation is done comparing ZPCodec against three baselines at the same 6 kbps bitrate: **Opus** (7), the standard real-time VoIP codec; **EnCodec** (2), designed for low-latency streaming; and **DAC** (4), a high-quality neural codec that employs non-causal convolutions and is therefore not suitable for real-time deployment. It is included as a quality upper bound for neural coding at this bitrate. All codecs are subjected to the exact same Gilbert-Elliott packet-loss patterns.

Table 2. Benchmark on Mozilla Common Voice test-set of 3994 samples; dataset not seen in train. **Loss** is the % of lost packets.

Loss	Codec	PESQ	P-Win	STOI	%-Win
10%	ZPCodec	**2.0920**	**50.33%**	**0.9017**	**60.48%**
	DAC	2.0874	48.90%	0.8913	39.42%
	EnCodec	1.7558	0.73%	0.8510	0.10%
	Opus	1.5780	0.05%	0.8174	0.00%
20%	ZPCodec	**1.8463**	**81.77%**	**0.8756**	**91.06%**
	DAC	1.5882	17.45%	0.8166	8.79%
	EnCodec	1.4617	0.78%	0.7804	0.15%
	Opus	1.3744	0.00%	0.7520	0.00%

5. Conclusions

In the end, the evaluation shows that Zero-Ping consistently outperforms all baselines across both perceptual quality and speech intelligibility, with the gap widening as loss rate increases. Apart from audio chat, the proposed approach extends naturally to any low-bitrate audio transmission scenario where packet loss is frequent, such as live streaming over congested networks or audio delivery in weak-signal environments, where waveform-level concealment consistently falls short. Finally, the 75 ms minimum chunk size is an artifact of Python invocation overhead: the current end-to-end latency totals $\sim$135 ms under packet loss, within the ITU-T G.114 150 ms. An optimized deployment (like in C++), as is standard for production codecs like Opus and EnCodec, would substantially reduce this overhead. The full project is available at https://github.com/Lucabr01/Zero-Ping. You can try it on the following notebook (CPU runtime is enough since the model is lightweight, consider using GPU only with large audio files): https://colab.research.google.com/drive/1S5OawwYIjd88uhVp1Vs7dog4gKcdvo-y?usp=sharing

Statement on the use of AI

The core usage was the exploration of literature, with strong focus on understanding all the parts related to signal/audio. Overall help with the code where i failed or needed support on implementing things like: the "connection" logic on quantization $\rightarrow$ transformer with the masks and the two-pass strategy (to fill the past embedding buffer at each "step"); tuning "mechanical" stuff for GPU optimization during training, for example in the joint training i used a RTX 5090 that required some code modifications. So in the architecture side, i entirely designed it and used AI just to help implementing whenever i needed support.

AI was heavily used on the evaluation side. In particular, it supported the implementation of the RTF benchmark and helped in understanding and interpreting the obtained results. During the benchmark implementation on the *Mozilla Common Voice* test set, AI was also heavily used to support audio preprocessing, format conversion, batching, and the adaptation of the evaluation pipeline to the different codec APIs (Opus, EnCodec, and DAC). In the notebook that can be used to try the model, the slider was entirely made by AI.

In the report i used it only for the mathematical formulism on the appendix.

References

[1] G. Haßlinger and O. Hohlfeld, "The gilbert-elliott model for packet loss in real time services on the internet," in *Proceedings of the 14th GI/ITG Conference on Measurement, Modelling and Evaluation of Computer and Communication Systems*, 2008, pp. 269–283.

[2] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," *arXiv preprint arXiv:2210.13438*, 2022. [Online]. Available: https://arxiv.org/abs/2210.13438

[3] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, "Soundstream: An end-to-end neural audio codec," *arXiv preprint arXiv:2107.03312*, 2021. [Online]. Available: https://arxiv.org/abs/2107.03312

[4] R. Kumar, P. Seetharaman, A. Luebs, I. Kumar, and K. Kumar, "High-fidelity audio compression with improved rvqgan," *arXiv preprint arXiv:2306.06546*, 2023. [Online]. Available: https://arxiv.org/abs/2306.06546

[5] H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson, "RTP: A transport protocol for real-time applications," RFC 3550, Jul. 2003. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3550

[6] ITU-T, "One-way transmission time," International Telecommunication Union, Recommendation G.114,

May 2003. [Online]. Available: https://www.itu.int/rec/T-REC-G.114

[7] J.-M. Valin, K. Vos, and T. Terriberry, "Definition of the opus audio codec," Internet Engineering Task Force, RFC 6716, Sep. 2012. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6716

Appendix

Loss Functions

Time-domain loss.

$$\mathcal{L}_{\text{time}} = \frac{1}{T}\sum_{t=1}^{T}|x_t - \hat{x}_t| \tag{1}$$

L1 loss on the raw waveform. Directly penalises amplitude errors sample by sample.

RVQ commitment loss.

$$\mathcal{L}_{\text{commit}} = \frac{1}{N_Q}\sum_{q=1}^{N_Q}\left(1 - \frac{\mathbf{r}_q \cdot \text{sg}(\mathbf{e}_q)}{\|\mathbf{r}_q\|\,\|\text{sg}(\mathbf{e}_q)\|}\right) \tag{2}$$

where $\text{sg}(\cdot)$ denotes stop-gradient, $\mathbf{e}_q$ is the nearest codebook entry at residual step q, and $\mathbf{r}_q$ is the current residual. Encourages the encoder to produce embeddings aligned with codebook entries in direction rather than magnitude, consistent with the cosine-similarity lookup used at quantisation time.

Latent repair loss.

$$\mathcal{L}_{\text{repair}} = \frac{1}{|\mathcal{M}| \cdot D}\sum_{t\in\mathcal{M}}\|\mathbf{z}_t^{\text{pre}} - \mathbf{z}_t^{\text{post}}\|_1 \tag{3}$$

where $\mathcal{M} = \{t : m_t = 0\}$ is the set of lost frames, $\mathbf{z}^{\text{pre}}$ is the original RVQ latent (target) and $\mathbf{z}^{\text{post}}$ is the Repair Transformer estimate. The mask restricts supervision to lost frames only; on received frames the two tensors coincide by construction.

Discriminator hinge loss.

$$\mathcal{L}_D = \frac{1}{K}\sum_{k=1}^{K}\Big(\mathbb{E}\big[\text{ReLU}(1-D_k(x))\big]+\mathbb{E}\big[\text{ReLU}(1+D_k(\hat{x}))\big]\Big) \tag{4}$$

where $K = 4$ is the number of STFT scales (windows $\in \{128, 256, 512, 1024\}$ samples) and D_k is the patch discriminator at scale k. Encourages real logits ≥ 1 and fake logits ≤ -1.

Multi-scale mel loss.

$$\mathcal{L}_{\text{mel}} = \frac{1}{S}\sum_{s=1}^{S}\Big(\|\mathbf{M}_s(x)-\mathbf{M}_s(\hat{x})\|_1+\|\log(\mathbf{M}_s(x)+\epsilon)-\log(\mathbf{M}_s(\hat{x})+\epsilon)\|_1\Big) \tag{5}$$

where $S = 7$ STFT scales (window lengths $\in \{32, \ldots, 2048\}$) and $\mathbf{M}_s$ is the amplitude mel-spectrogram at scale s. The linear term emphasises energetic peaks (formants, plosives); the log term preserves weaker components (fricatives).

Generator adversarial loss.

$$\mathcal{L}_{\text{adv}} = -\frac{1}{K}\sum_{k=1}^{K}\mathbb{E}\big[D_k(\hat{x})\big] \tag{6}$$

Pushes the generator to produce reconstructions that the discriminator scores as real.

Feature matching loss.

$$\mathcal{L}_{\text{fm}} = \frac{1}{KL}\sum_{k=1}^{K}\sum_{l=1}^{L}\big\|\phi_k^{(l)}(x) - \phi_k^{(l)}(\hat{x})\big\|_1 \tag{7}$$

where $\phi_k^{(l)}$ denotes the l-th intermediate feature map of discriminator k. Real feature maps are detached, so gradients flow only through the generator. Stabilises GAN training by encouraging the generator to match the internal representations of real audio at every layer.

Training Stages

RVQ only: The encoder, quantizer, and decoder are trained jointly with the full generator objective $\mathcal{L}_G = \mathcal{L}_{\text{time}}+\lambda_{\text{mel}}\,\mathcal{L}_{\text{mel}}+\lambda_{\text{commit}}\,\mathcal{L}_{\text{commit}}+\lambda_{\text{adv}}\,\mathcal{L}_{\text{adv}}+\lambda_{\text{fm}}\,\mathcal{L}_{\text{fm}}$. The repair Transformer is not involved. This stage establishes a stable latent space and trains the discriminator before any concealment is introduced.

Repair Transformer only: The codec and discriminator weights are frozen. Only the Transformer is trained using $\lambda_{\text{rep}}\mathcal{L}_{\text{repair}} + \lambda_{\text{mel}}\,\mathcal{L}_{\text{mel}}$, where $\mathcal{L}_{\text{repair}}$ supervises the reconstructed latents on lost frames only and $\mathcal{L}_{\text{mel}}$ provides an auditory quality signal on the decoded output. Freezing the codec ensures the latent space is not perturbed while the repair module learns to exploit it.

Joint fine-tuning: All components are unfrozen and trained end-to-end with the same objective as Stage 1, i.e. $\mathcal{L}_G$ without $\mathcal{L}_{\text{repair}}$. Dropping the latent supervision encourages the system to optimise perceptual quality directly, allowing the encoder, decoder, and Repair Transformer to adapt jointly under the adversarial objective.